Vision Based State Estimation

EECS 116/216 Fall 2026 | Project 2

UC Berkeley

Complete the project in your assigned **groups of 2–3**, as in Project 1.

Estimate a quadrotor's pose and velocity from its downward-facing camera. Use the supplied AprilTag detections for pose and track image corners for velocity. Mocap measurements are provided **only for comparison**; do not use them or the recorded IMU readings as estimator inputs.

- **Part 1**
  Measurement inputs: AprilTag IDs and image corners.
  Estimated quantities: Body position in the world frame and body-to-world orientation (six pose degrees of freedom).
- **Part 2**
  Measurement inputs: Consecutive images, timestamps, and the pose estimated in Part 1.
  Estimated quantities: Body-origin linear velocity in the world frame and angular velocity in the body frame (three components each).

1 Vision Based Pose Estimation

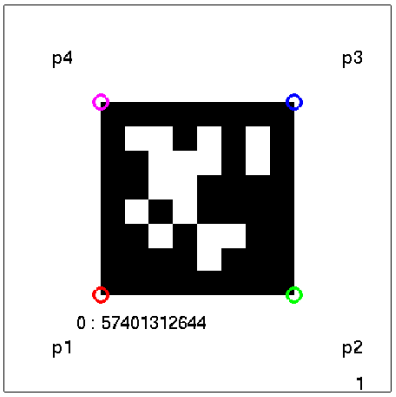
1.1 Environment

The Nano+ quadrotor was held or flown over a mat of AprilTags and random scribble. The tags form a 12-row, 9-column grid. The world origin is the top left corner of the top left tag, with $+X$ down the mat, $+Y$ to the right, and $+Z$ upward from the floor. These are right-handed axes. With zero-based row i and column j, the tag ID is $12j + i$.

Each tag is a 0.152 m square. The edge-to-edge gap between tags is 0.152 m, except between columns 3 and 4 and columns 6 and 7 (counting from 1), where it is 0.178 m. All tag corners lie in the world $Z = 0$ plane.



AprilTag mat and world axes.



Corner order: bottom left, bottom right, top right, top left (p_1, p_2, p_3, p_4).

1.2 Calibration

The supplied `support.py` defines the intrinsic matrix

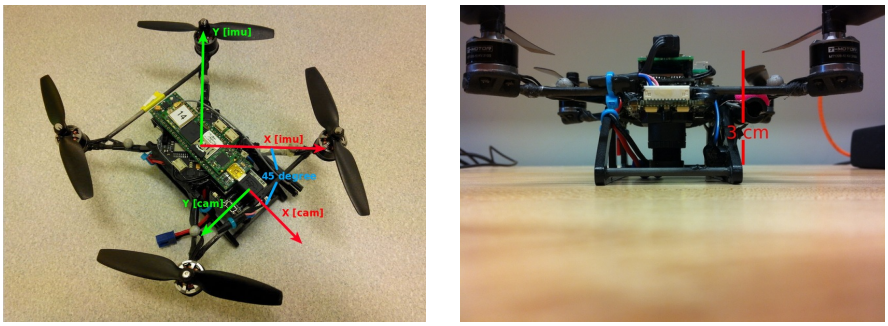
$$K = \begin{bmatrix} 311.0520 & 0 & 201.8724 \\ 0 & 311.3885 & 113.6210 \\ 0 & 0 & 1 \end{bmatrix}.$$

Image coordinates (u, v) are **zero-based pixels**, with u to the right and v downward. Images and tag detections are already rectified; do not apply another distortion correction. The camera axes are $+x$ right, $+y$ down, and $+z$ forward along the optical axis.

Use the supplied camera–body calibration with the explicit convention

$$\mathbf{p}_C = R_{CB}\mathbf{p}_B + \mathbf{t}_{CB}, \qquad R_{CB} = R_z(-\pi/4)R_x(\pi), \qquad \mathbf{t}_{CB} = \begin{bmatrix} -0.04 & 0 & -0.03 \end{bmatrix}^T \text{ m}.$$

`BODY_TO_CAMERA` is the corresponding 4×4 homogeneous transform. The translation is the *body origin expressed in camera coordinates*. For this exercise the body, IMU, and tracked mocap frames coincide; the camera origin is offset. Account for both rotation and translation when converting the camera pose or velocity to the body pose or velocity.



Camera mounting: top view and side view.

1.3 Data and Pose Estimation

The supplied loader reads `data/studentdata1.mat` and `data/studentdata4.mat`. It returns image frames with:

```
time     # Image timestamp, seconds
image    # Rectified grayscale image (height, width)
ids      # Detected tag IDs (N,)
corners  # Pixels (N, 4, 2): p1, p2, p3, p4 for each tag
```

Complete `get_corners()` and `estimate_pose()` in `pose.py`. Compute world corner coordinates, estimate a planar homography by SVD, and recover the camera pose using K. NumPy's SVD returns V^T as its third result; a single tag gives an 8×9 system, so retain the full right nullspace. Choose the homography sign giving *positive camera depths* and recover a proper rotation with determinant $+1$. Transform the result to a `Pose`: world-frame body position and body-to-world rotation matrix. Return `None` when no tags are visible.

The loader also returns `reference_time` and the original `vicon` matrix, whose rows are

$$[x, y, z, \text{roll}, \text{pitch}, \text{yaw}, v_x, v_y, v_z, \omega_x, \omega_y, \omega_z]^T.$$

Position and linear velocity are world-frame quantities (m, m/s); angular velocity is body-frame (rad/s). Orientation uses intrinsic ZYX rotations, $R_{WB} = R_z(\text{yaw})R_y(\text{pitch})R_x(\text{roll})$. The plots and recorded angles are ordered **roll, pitch, yaw**, in radians. Image and mocap streams retain their own timestamps; the plotter overlays them without assuming equal sample counts.

2 Corner Extraction and Tracking

Complete `estimate_velocity()` in `flow.py`. Detect image corners, including those in the random scribble, and compute sparse optical flow between consecutive images with a Kanade–Lucas–Tomasi (KLT) tracker. You may use OpenCV's corner detectors and `calcOpticalFlowPyrLK()`; see the OpenCV optical-flow tutorial. Initialize points in the *previous* image, track them into the current image, and discard unsuccessful tracks. Assume all retained features lie on the ground plane.

Calibrate the pixel coordinates using K. Divide normalized image displacements by the elapsed image time to obtain $[\dot{x}, \dot{y}]^T$ in s^{-1}. Timestamp jitter introduces velocity noise; discuss its effect on your results. The provided runner supplies the previous image's pose for depth and frame calculations and plots each interval's velocity at its midpoint timestamp.

2.1 Velocity Estimation

For a static point with normalized coordinates (x, y) and camera depth Z, derive the image-motion equations

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \begin{bmatrix} \mathbf{f}_1(x, y, Z) \\ \mathbf{f}_2(x, y, Z) \end{bmatrix} \begin{bmatrix} V_x & V_y & V_z & \Omega_x & \Omega_y & \Omega_z \end{bmatrix}^T.$$

Each $\mathbf{f}_i$ is a 1×6 row. Here V is the velocity of the *camera origin* and Ω is camera angular velocity, both expressed in the camera frame. Obtain each feature's depth by intersecting its ray with the world $Z = 0$ plane using the previous camera pose. Camera depth is neither body height nor distance along the ray; it is the point's camera-z coordinate.

Implement the linear motion fit in `velocity_ransac()`. Then transform the fitted camera motion into the requested body-origin world-frame linear velocity and body-frame angular velocity. Rotation about the body origin moves the offset camera; include this effect. Return `None` when fewer than three usable tracks remain; the plotter leaves a gap for missing estimates.

2.2 RANSAC-Based Outlier Rejection

Implement three-point RANSAC for the motion fit. Sample three distinct point correspondences per trial, reject rank-deficient systems, and classify each point using the norm of its two-component optical-flow residual. Keep the best consensus and refit using all its inliers. The function receives the residual threshold and iteration count as inputs. Choose these values and the detector/tracker parameters in `estimate_velocity()` and explain your choices. With RANSAC disabled, fit all usable tracks by least squares.

2.3 Implementation and Running

Edit only `pose.py` and `flow.py`; you may add helpers in these files. Preserve the supplied signatures, return types, calibration, runner, and data. Use NumPy, SciPy, OpenCV, and the supplied helpers; **no other dependencies**. Implement homography estimation/decomposition, the velocity fit, and RANSAC yourself; library corner detectors and KLT tracking are permitted.

Use the Python 3.10 `eecs116` conda environment from Project 1 on the Ubuntu 22.04 lab machines. In the folder containing `main.py`, run:

```
conda activate eecs116
python -m pip install -r requirements.txt
python main.py --part 1 --dataset 1
python main.py --part 2 --dataset 1 --ransac off
python main.py --part 2 --dataset 1 --ransac on
```

Repeat all three runs with `--dataset 4`.

3 Report

Submit a PDF report prepared with LaTeX or Word, at most **8 pages including plots**. List all group members' names and UC Berkeley student IDs at the top. Explain your pose and velocity models, frame transformations, implementation, and parameter choices. Include the supplied plots for both datasets: pose, velocity without RANSAC, and velocity with RANSAC. Discuss agreement with the mocap measurements and the effect of outlier rejection. Support your equations with an explanation of why and how you used them.

4 Grade Policy and Submission

The total is **100 points**: pose estimation (40), velocity estimation (30), RANSAC (20), and report/code quality and readability (10). Each call to `run_estimation()` must finish within one minute per dataset on the lab machines, measured in wall-clock time and excluding data loading and plotting. Runs exceeding this limit will not be graded. Changes outside the permitted files receive zero points.

Put `report.pdf` alongside `main.py`. Submit one archive named `submission.zip` with exactly this structure:

```
submission.zip
|-- report.pdf
|-- main.py
|-- support.py
|-- pose.py
|-- flow.py
|-- requirements.txt
`-- data/
    |-- studentdata1.mat
    `-- studentdata4.mat
```

The report and Python files must be **directly at the ZIP's top level**, not inside an enclosing `student/` or group folder. Include the supplied data unchanged. Do not include your Git repository, Python environment, caches, or separate plot files; include plots in the report.

From the folder containing `main.py` and `report.pdf`, run:

```
python -m zipfile -c ../submission.zip report.pdf main.py \
  support.py pose.py flow.py requirements.txt data
python -m zipfile -l ../submission.zip
```

The first command creates the archive in the parent folder, replacing any previous archive with that name. Check the listed contents before submitting. After extraction, the code must run without moving files or changing paths.

Upload `submission.zip` to bCourses. **Each group member must submit the same ZIP using their own account.**